

Master's Thesis

Adaptive Rounding for Efficient Post-Training Quantization of Deep Neural Networks

48246347 Piye Liu

Supervisor Prof. Hiroshi Nakamura

August, 2026

Department of Information Physics and Computing
Graduate School of Information Science and Technology
The University of Tokyo

概要

Abstract in Japanese should be written here.

Abstract

300 words abstract in English should be written here.

Contents

Chapter 1	Introduction	1
1.1	Background	1
1.2	Problem Formulation	3
1.3	Contributions	3
Chapter 2	Preliminary	5
2.1	Compressibility of Deep Neural Networks	5
2.2	Small Language Models	8
2.3	Baseline Round-to-Nearest Quantization	10
2.4	Quantization is Weight Perturbation	11
2.5	Effects of Perturbation of Model Weights	11
Chapter 3	Literature Review	16
3.1	AdaRound	16
Chapter 4	Proposed Method	19
4.1	AdaRound Inversion Method	19
4.2	Gradient-Aware Adaptive Rounding	20
Chapter 5	Evaluation	22
5.1	AdaRound Inversion Method	22
5.2	Gradient-Aware Adaptive Rounding	26
Chapter 6	Conclusion	28
	Bibliography	29

Chapter 1

Introduction

1.1 Background

1.1.1 Bias-Variance Tradeoff and Scaling Laws

Classical Perspective

Consider a classifier over $k \in \{1, 2, \dots, K\}$, trained with Mean Squared Error loss. Its test error is often attributed to the following three sources [1].

$$\mathcal{E}_{\text{total}} = \mathcal{E}_{\text{noise}} + \mathcal{E}_{\text{bias}} + \mathcal{E}_{\text{variance}} \quad (1.1)$$

The first term is an intrinsic error term representing irreducible noise, the second term measure the systematic error the model makes, and the third term captures the variance in the model's predictions. As the capacity of a model increases, the bias decreases while the variance increases, leading to a parabolic test error curve [2]. It is said that:

The price to pay for achieving low bias is high variance. — Geman et. al., 1992 [1]

This means that if the model's capacity is too small, it may underfit the training data and perform poorly on unseen data. If the model's capacity is too large, it may overfit to spurious patterns in the training data, resulting in low accuracy on unseen data anyways [3]. Textbooks even assert that "a model with zero training error is overfit to the training data and will typically generalize poorly" [4, pg. 221]. It was therefore common to carefully select the complexity of models to achieve the optimal balance between bias and variance.

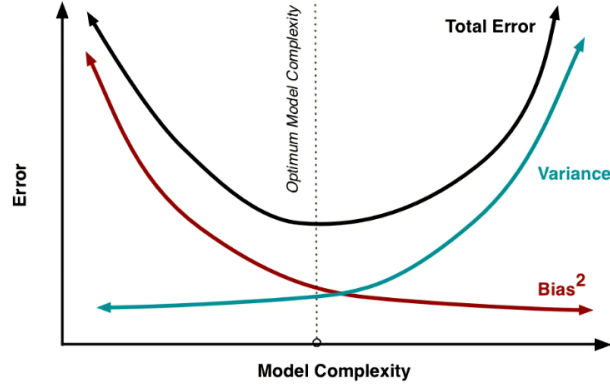


Fig. 1.1: Bias-Variance Tradeoff [2]

Modern Perspective

Modern machine learning, however, regularly uses models with low or zero training error, and those models often give very accurate predictions on new data [3]. It was then proposed to reconcile this phenomenon and the classical bias-variance tradeoff using a “double descent” curve, as shown in Figure 1.2.

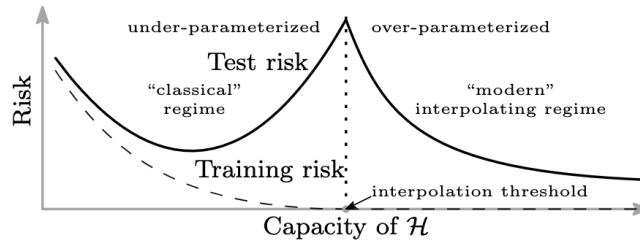


Fig. 1.2: Double Descent Curve [3]

When originally proposed in 2019, this phenomenon was empirically validated on neural networks and random forests on the MNIST dataset [3]. But importantly, it was later empirically validated on a transformer-based large language model with up to 800M parameters [5]: we can clearly observe a double-descent curve in Figure 1.3. The authors also propose the theory that the peak in test loss in the middle of the learning curve is the model memorizing the training data, while the continued descent afterwards is the model actually learning from the training data.

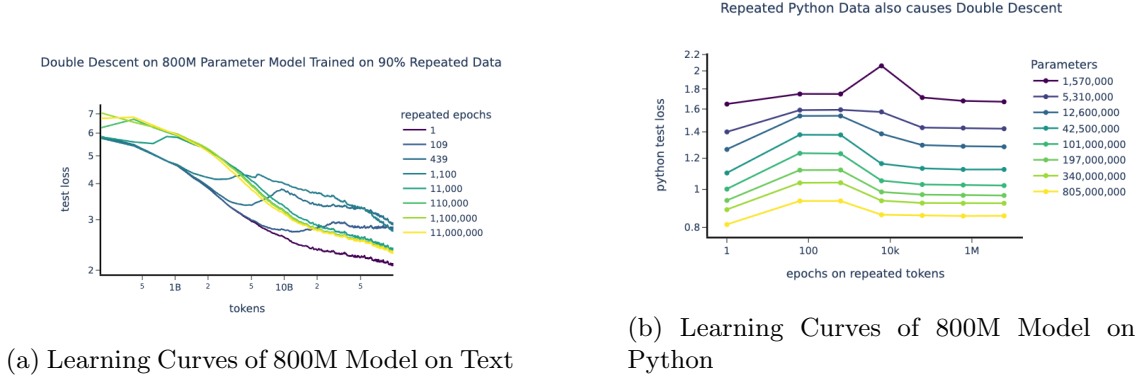


Fig. 1.3: Double Descent Phenomenon on Transformer-based Language Models [5]

Seemingly, simply adding more training data and parameters allows one to create better language models. This phenomenon has led to an exponential scaling of LLM parameter counts, as surveyed in Figure 1.4.

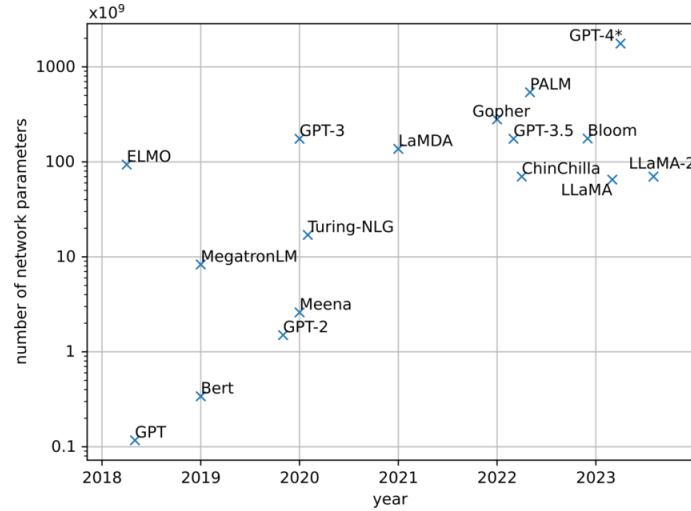


Fig. 1.4: Observed exponential scaling of LLM Parameters over time [6](Note: The GPT-4 figure is an estimate)

1.2 Problem Formulation

1.2.1 Motivation

1.2.2 Survey of Compression Methods

1.3 Contributions

Our contributions in this work are twofold: First, we demonstrate a novel inversion method against AdaRound to recover coarse outlines that represent its calibration data, motivating

4 Chapter 1 Introduction

the need to consider privacy in data-dependent quantization method. Second, we propose a gradient-aware adaptive rounding method based on AdaRound, and empirically validate the proposed method on two language models.

Chapter 2

Preliminary

2.1 Compressibility of Deep Neural Networks

In this section, we describe a simple way to quantify the compressibility of models. The source coding theorem [7] states that:

- n i.i.d. (independent and identically distributed) random variables, each with entropy $H(X)$, can be compressed to $nH(X)$ bits on average, with negligible information loss, as $n \rightarrow \infty$
- Conversely, if they are compressed to fewer bits on average, loss of information must occur.

In this case, $H(X)$, the entropy of a random variable, is the expected amount of information needed to describe it, and can be computed by the following.

$$H(X) := - \sum_{x \in X} p(x) \log_2 p(x) \quad (2.1)$$

Note that the log base determines the units: $\log_2$ yields bits, $\log_{10}$ yields dits, and $\ln$ yields nats.

Consider a model with weights W , each stored in b bits, and there are n weights in total. The total number of bits occupied by the model is therefore $n \cdot b$. This is the capacity of the model: it is the maximum amount of information the weights could carry if every possible bit pattern were equally likely and the weights were statistically independent.

In practice, however, models rarely use their full capacity. We can measure this by treating its weights as independent draws from a categorical distribution over these possible values.

$$\mathcal{X} = \{0, 1, \dots, 2^b - 1\} \quad (2.2)$$

We can use Maximum Likelihood Estimation (MLE) to estimate the distribution as

follows.

$$\hat{p}(x) = \frac{1}{n} \sum_{k=1}^n \mathbf{1}(W_k = x), \text{ where } \mathbf{1}(p) = \begin{cases} 1 & \text{if } p \\ 0 & \text{if } \neg p \end{cases} \quad (2.3)$$

And we can estimate the entropy per weight as follows.

$$\hat{H}(W) := - \sum_{x \in \mathcal{X}} \hat{p}(x) \log_2 \hat{p}(x) \quad (2.4)$$

Note that if a certain value does not appear at all in the model weights, there will be an undefined term $0 \ln 0$ in the above calculation. We use a convention in modern information theory to mitigate this, by using an extension of $x \ln x$ in the form of the following [8, eq. 1.1.2].

$$f(x) = \begin{cases} 0, & x = 0 \\ x \ln x, & x > 0 \end{cases} \quad (2.5)$$

This allows us to treat $0 \ln 0$ as 0. We can see that the total information stored in the weights would be approximately $n \cdot \hat{H}(W)$ bits. But the maximum possible total entropy of the model is $n \cdot b$ bits (achieved by the uniform distribution), it therefore follows that the capacity utilization by a model is the following.

$$U(W) := \frac{\hat{H}(W)}{b} \quad (2.6)$$

Note that $U \in [0, 1]$, and $U = 1$ implies the model is nearly incompressible losslessly, while $U < 1$ implies the model can be losslessly compressed to $1 - U$ times its current size. For example, if $U = 0.5$, the model can be compressed to half its size with negligible information loss.

Qwen 3 [9] is a state-of-the-art language model in various sizes. Now that we have a method to quantify the compressibility of models, let us evaluate the compressibility of the 4B model, which is provided in three data types, of widths 16, 8, and 4. We report the results in Table 2.1.

Consider reproducing with more models

Table 2.1: Entropy Estimate of Qwen 3 4B Weights

Data Type	$\hat{H}(W)$ in bits	$U(W)$
bfloat16	10.58	$10.58/16 \approx 66\%$
fp8	6.63	$6.63/8 \approx 82.9\%$
int4	3.67	$3.67/4 \approx 91.8\%$

We observe that the bfloat16 model, which is ~ 8 GB on disk, only uses $\sim \frac{2}{3}$ of its capacity. This implies that the model can be compressed to ~ 5.33 GB with negligible information loss. This degree of compressibility motivates us to perform an investigation

of the distribution of the weights by plotting their possible values against the probabilities of each value, which we report in Figures 2.1, 2.2, and 2.3.

No axis labels (need to rerun the code to add them)

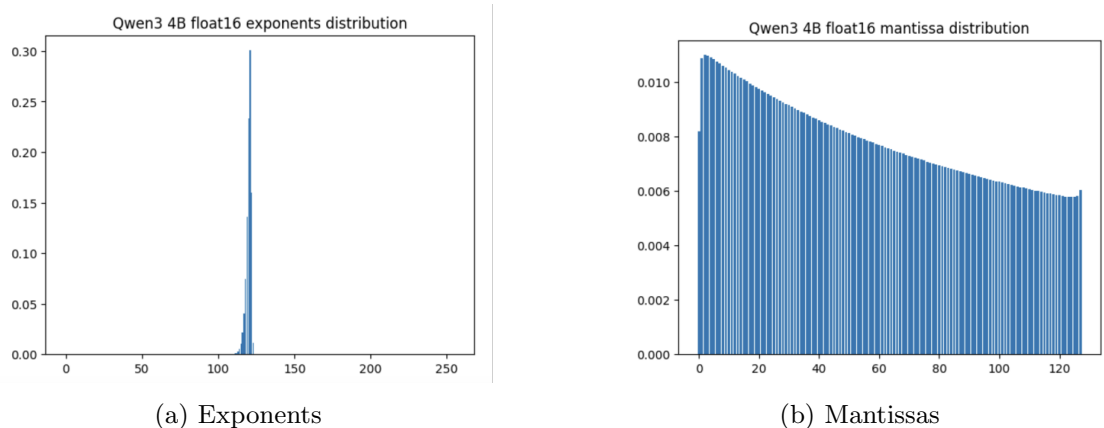


Fig. 2.1: Distribution of Qwen3-4B Weights (bfloat16)

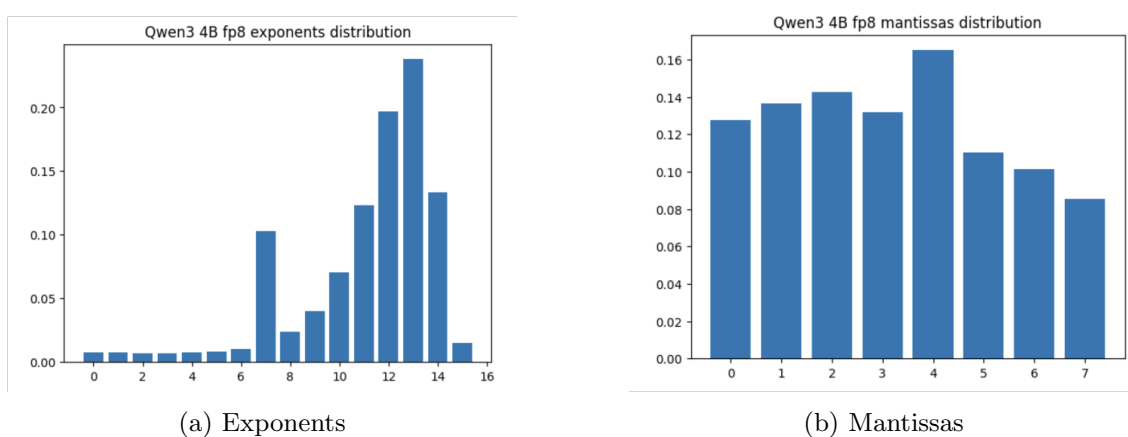


Fig. 2.2: Distribution of Qwen3-4B Weights (fp8)

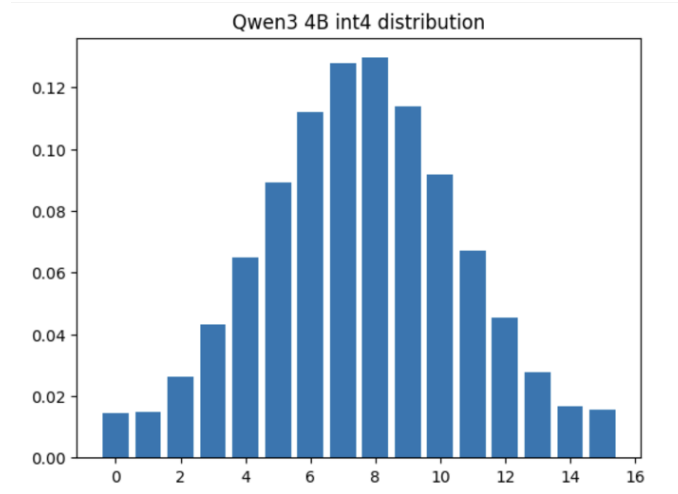


Fig. 2.3: Distribution of Qwen3-4B Weights (int4)

We notice that the full dynamic range of bfloat16 is actually not used. Especially in bfloat16, the exponents are clustered towards the center, with an average of 120 (this means most weights are $m \cdot 2^{-7}$). Several factors may contribute to this phenomenon: weight decay, regularization, and the optimizer (for example, stochastic gradient descent provides implicit optimization [10]).

We therefore conclude that large language models often have much room for compression, and is not using all of its potential capacity. This provides motivation for research in compression methods.

2.2 Small Language Models

In this section, we will examine TinyStories [11], a work that pushes the boundaries of small language models, and provide a few custom language models for further experimentation. By using a low-variance synthetic dataset (children’s stories) generated by GPT-3.5 and GPT-4 containing only words a typical 3-4 year old would understand, they were able to generate coherent output using models with as few as 1 million parameters.

One limitation we observed in this work is their usage of the default GPT-Neo [12] tokenizer (which is identical to the pretrained GPT-2 [13] tokenizer), restricted to the top 10,000 tokens. Using a generic tokenizer for a low-vocabulary dataset like TinyStories might result in wasted weights (tokens that never appear in TinyStories), and might not provide the optimal tokenization for our specialized dataset.

As a preliminary experiment, we train two small language models on TinyStories v1 and TinyStories v2, respectively. We use a SentencePiece Byte-Pair Encoding (BPE) tokenizer [14] with a vocabulary size of 1024, and we used a LLaMa-based [15] model with roughly half the parameters of the smallest reference model.

2.2.1 Evaluation: Quantitative Metric

We use perplexity [16] as a quantitative metric to evaluate a language model’s accuracy. For a model θ , a text x of length N , it is defined as the following.

$$\text{Perplexity}(\theta, x) = \left(\prod_{i=1}^N \frac{1}{p_{\theta}(x_i \mid x_1, x_2, \dots, x_{i-1})} \right)^{\frac{1}{N}} \quad (2.7)$$

In other words, it is the geometric mean of the inverse probabilities of a given text. To show this with an example, suppose we have a model θ and a ground truth text “Mary had a little lamb”. Suppose that the model’s predicted probabilities of this text is the following.

Table 2.2: Example Model Output Probabilities

Item	Value
$p_{\theta}(\text{“Mary”})$	0.5
$p_{\theta}(\text{“had”} \mid \text{“Mary”})$	0.7
$p_{\theta}(\text{“a”} \mid \text{“Mary”, “had”})$	0.9
$p_{\theta}(\text{“little”} \mid \text{“Mary”, “had”, “a”})$	0.6
$p_{\theta}(\text{“lamb”} \mid \text{“Mary”, “had”, “a”, “little”})$	0.9

Then the perplexity of the model on the text is the following.

$$\text{Perplexity} = \left(\frac{1}{0.5} \cdot \frac{1}{0.7} \cdot \frac{1}{0.9} \cdot \frac{1}{0.6} \cdot \frac{1}{0.9} \right)^{\frac{1}{5}} \quad (2.8)$$

$$\approx 1.49 \quad (2.9)$$

Intuitively, perplexity is a measure of how surprised or confused a model is when seeing a certain text. We compute the model’s perplexity over its validation data, then use the result as a metric to evaluate the model’s accuracy (lower is better).

2.2.2 Evaluation: Results and Discussion

We report the validation perplexity of our two models. Note that the original paper only provided models trained on TinyStories v1.

Table 2.3: Evaluation of Our Language Models

	Reference	Ours (TinyStories v1)	Ours (TinyStories v2)
Architecture	GPT-Neo	LLaMa	LLaMa
Parameter Count	1.17M	0.57M	0.46M
Training Data	TinyStories v1		TinyStories v2
Tokenizer (Vocab Size)	GPT-Neo	SentencePiece	SentencePiece
Vocab Size	10000	1024	1024
Perplexity (TinyStories v1)	19.3919	11.9006	—
Perplexity (TinyStories v2)	—	—	8.0994

Add qualitative results (actual model outputs) here

Consider an ablation study to train the same GPT-Neo model, with only a different tokenizer

We observe that our model results in better perplexity than the reference model despite being much smaller. In `float32`, each weight is 4 bytes, so our model is expected to take only $\sim 2.17\text{MiB}$ of disk space (the actual disk space, including the model’s configuration, is $\sim 2.22\text{MiB}$). Models of this scale can be suitable for devices with extremely low compute and/or memory.

We will use our models as a base to conduct several more experiments.

2.3 Baseline Round-to-Nearest Quantization

The baseline for post-training quantization is symmetric uniform quantization using round-to-nearest (RTN) rounding [17]. A model can be quantized to b bits using Algorithm 1, then dequantized using Algorithm 2.

Algorithm 1 Baseline RTN Quantization

Require: W (unquantized weights), b (target bit width)

Ensure: Quantized model W_q

- 1: $q_{\min} \leftarrow -2^{b-1}$
 - 2: $q_{\max} \leftarrow 2^{b-1} - 1$
 - 3: $\text{scale} \leftarrow \frac{\max(W)}{q_{\max}}$
 - 4: $W_q \leftarrow \text{clamp}(\lceil \frac{W}{\text{scale}} \rceil, q_{\min}, q_{\max})$
 - 5: **return** W_q, scale
-

Algorithm 2 Baseline RTN Dequantization

Require: W_q (quantized weights), scale

Ensure: Dequantized weights $\hat{W}$

- 1: $\hat{W} \leftarrow W_q \cdot \text{scale}$
 - 2: **return** $\hat{W}$
-

2.4 Quantization is Weight Perturbation

Let us establish the observation that quantization results in a perturbed model, using an example. Consider a model θ with parameters $W = \begin{bmatrix} 1.11 \\ 2.22 \\ 3.33 \end{bmatrix}$. By applying the baseline round-to-nearest quantization from Section 2.3, we obtain the following quantized model.

$$\text{scale} \approx 0.0262, W_q = \begin{bmatrix} 0b00101010 \\ 0b01010101 \\ 0b01111111 \end{bmatrix} = \begin{bmatrix} 42 \\ 85 \\ 127 \end{bmatrix} \quad (2.10)$$

After dequantization, we obtain the following model.

$$W_{\text{reconstructed}} = \text{scale} \cdot W_{\text{quantized}} \approx \begin{bmatrix} 1.1013 \\ 2.2287 \\ 3.33 \end{bmatrix} \quad (2.11)$$

We can see that the model is perturbed by the following.

$$\Delta W = \begin{bmatrix} 1.1013 \\ 2.2287 \\ 3.33 \end{bmatrix} - \begin{bmatrix} 1.11 \\ 2.22 \\ 3.33 \end{bmatrix} = \begin{bmatrix} -0.0087 \\ 0.0087 \\ 0 \end{bmatrix} \quad (2.12)$$

It is important to note the direction of perturbation. In this case, our model moved in the direction of $\begin{bmatrix} -1 \\ 1 \\ 0 \end{bmatrix}$.

2.5 Effects of Perturbation of Model Weights

As we established in Section 2.4, quantization perturbs weights in a certain direction. In this section, we examine the effects of weight perturbation using our TinyStories v2 model, by viewing its loss landscape.

2.5.1 Language Model Optimization Process

To understand our model's loss landscape, we must first understand its loss function. The goal in optimizing language models is to minimize the difference between the ground truth and the model's predictions. We measure this using the Kullback-Leibler (KL) divergence [18], defined as follows.

$$D_{KL}(P \parallel Q) = \sum_{x \in X} p_P(x) \ln \frac{p_P(x)}{p_Q(x)} \quad (2.13)$$

The KL-divergence measures how an approximating distribution Q is different from a distribution P . We use one-hot encoding to encode the ground truth into the distribution P , and use the model's outputs as the approximating distribution Q .

Note that we once again need to employ the convention in Equation 2.5. Let us show why with an example: if we are predicting a word “Hello”, the ground truth P distribution and the predicted Q distribution might be:

Table 2.4: Example of KL-divergence

$\hat{y}$	$p_P(\hat{y})$	$p_Q(\hat{y})$
“Today”	0	0.07
“Hello”	1	0.81
“Cat”	0	0.10
“Rain”	0	0.02

In this case, the KL-divergence is:

$$D_{KL}(P \parallel Q) = \underbrace{0 \cdot \ln \frac{0}{0.07}}_{\text{“Today”}} + \underbrace{1 \cdot \ln \frac{1}{0.81}}_{\text{“Hello”}} + \underbrace{0 \cdot \ln \frac{0}{0.10}}_{\text{“Cat”}} + \underbrace{0 \cdot \ln \frac{0}{0.02}}_{\text{“Rain”}}, \quad (2.14)$$

which contains three undefined terms in the form of $0 \cdot \ln(\frac{0}{n})$. Using the convention from Equation 2.5, we can compute the undefined terms as follows.

$$0 \cdot \ln \frac{0}{n} = \underbrace{0 \ln 0}_{0 \text{ by convention}} - 0 \ln n = 0 \quad (2.15)$$

We then obtain the following.

$$D_{KL}(P \parallel Q) = -\ln(0.81) \approx 0.21 \quad (2.16)$$

When minimizing the KL-divergence between the model predictions and the ground truth, we can use the following observation to ignore the constant $H(P)$ term, or the entropy of the ground truth distribution P .

$$D_{KL}(P \parallel Q) = \underbrace{\sum_{x \in X} p_P(x) \ln p_P(x)}_{-H(P)} - \sum_{x \in X} p_P(x) \ln p_Q(x) \quad (2.17)$$

$$\propto - \sum_{x \in X} p_P(x) \ln p_Q(x) \quad (2.18)$$

When optimizing, we only need to minimize $-\sum_{x \in X} p(x) \ln q(x)$. This is also known as the cross entropy $H(P, Q)$ between distributions P and Q .

The cross entropy for a language model θ on a single token, where P_i is the one-hot

encoded ground truth and Q_i is the model’s predicted distribution, is the following.

$$H(P_i, Q_i) = - \sum_{x \in X} p_{P_i}(x) \ln p_{Q_i}(x) = - \ln p_{\theta}(x_i \mid x_{<i}) \quad (2.19)$$

The loss function, the average cross-entropy per token over a sequence of length N , is the following.

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N H(P_i, Q_i) = - \frac{1}{N} \sum_{i=1}^N \ln p_{\theta}(x_i \mid x_{<i}) \quad (2.20)$$

Note that this is exactly the natural logarithm of the perplexity of the model on the given text. Given this loss function, we will proceed to investigate the loss landscape of our model.

2.5.2 Loss Landscape

Since the loss landscape of a language model is in n -dimension, where n is its number of parameters, it is difficult to be directly visualized. We will investigate the loss landscape of our TinyStories v2 model from Section 2.2 by perturbing its weights and computing the perplexity of the perturbed models.

Straight Line

First, we will sample models along a simple line in the following way.

$$\theta' = \theta + \epsilon, \epsilon \in \{-0.0020 \dots 0.0015\} \quad (2.21)$$

Note that the blue line in the figure represents $\epsilon \in \{-0.0015 \dots 0.0015\}$. From the plot in Figure 2.4, we notice that there is a “basin” of low loss, and that moving towards the negative direction caused less degradation than moving towards the positive direction.

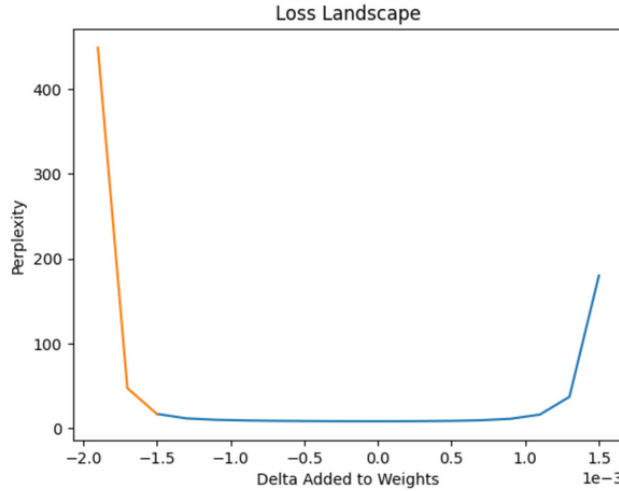


Fig. 2.4: Models Along a Line

Moving Randomly

Next, we sample models stochastically from the following distribution. We sample 5 models for each σ .

$$\theta' \sim \mathcal{N}(\theta, \sigma^2 \mathbf{I}), \sigma^2 \in \{0.0005 \dots 0.0040\} \quad (2.22)$$

Note that we cap the perplexity at 100 in Figure 2.5 for better visualization, because some perplexities are extremely large.

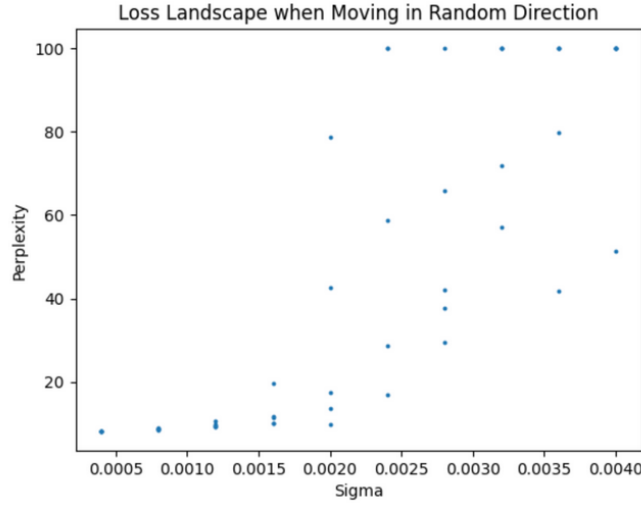
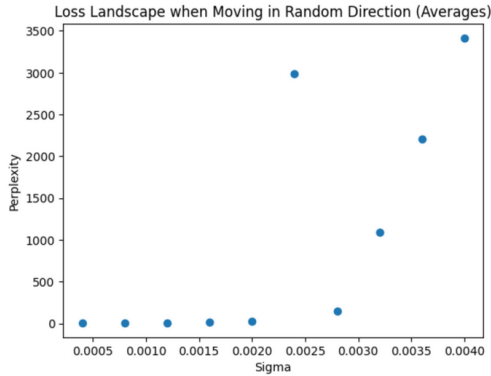
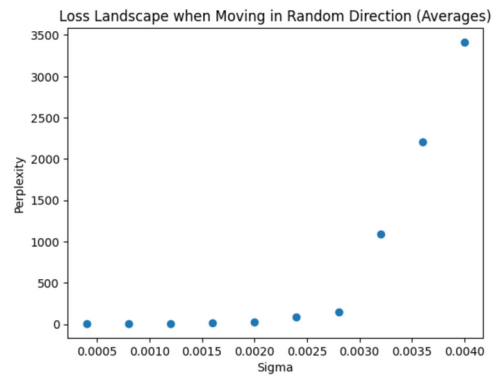


Fig. 2.5: Models from Gaussian Distribution

In Figure 2.6a, we plot the average perplexities of each σ . We notice an outlier at $\sigma = 0.0024$, and removing it yields Figure 2.6b.



(a) Averages



(b) Averages (Outlier Removed)

Fig. 2.6: Loss Landscape of Models from Gaussian Distribution

This further confirms the observation that when perturbing a model, the direction of perturbation is important, and some directions yield terrible results (like the outlier) while

some directions preserve the model's accuracy more.

Towards Another Model

We pretrain another model with the same configuration as the base model, and move from the base model to the other model. The distance between these models, $\|\theta - \theta'\|_2$, is 75.6, which means they're quite far apart, and the magnitude of perturbation is much greater than the previous experiments.

Note that when the x-axis is 0, the model is exactly the base model, and when the x-axis is 1, the model is exactly the other pretrained model.

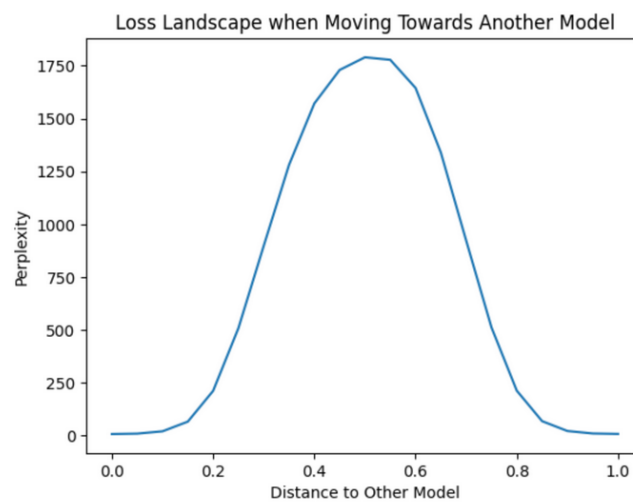


Fig. 2.7: Moving Towards Another Model

We observe from Figure 2.7 that even though the magnitude of perturbation is much greater than the previous experiments, the increase in loss is not as significant. This shows that the model is less sensitive to perturbations in this direction.

2.5.3 Discussion

The key insight from this experiment is the fact that

Consider adding an experiment: loss when moving towards the direction of the gradient

Write how some directions are better than others, and worst directions cause significant degradation

Chapter 3

Literature Review

3.1 AdaRound

3.1.1 Notation

Let us establish the notation for variables in Table 3.1.

Table 3.1: Notation for this Literature Review

Symbol	Description
w_j	Weights of a layer j
x_j	Input to a layer j
b	Target bit-width quantization
scale	Continuous scaling factor
$\lceil \cdot \rceil$	Round-to-nearest-integer operator
$\lfloor \cdot \rfloor$	Floor operator
$\mathbf{W}, \hat{\mathbf{W}}$	Matrices of unquantized and quantized weights
$\mathcal{L}$	A loss function

3.1.2 Introduction to AdaRound

AdaRound [19] is a novel rounding method, based on the key insight that round-to-nearest is suboptimal for post-training quantization (PTQ).

The authors’ proposed method is to instead minimize the difference in task loss of the quantized model and the unquantized model. The task loss of layer j of the unquantized model is $\mathcal{L}(x, y, w_j)$, and the task loss of the same layer of the quantized model $\mathcal{L}(x, y, w_j + \Delta w_j)$. Therefore, the authors’ objective is the following.

$$\arg \min_{\Delta w_j} (\mathcal{L}(x, y, w_j + \Delta w_j) - \mathcal{L}(x, y, w_j)) \quad (3.1)$$

The authors use a second order Taylor expansion to approximate the above objective.

$$\mathcal{L}(x, y, w_j + \Delta w_j) - \mathcal{L}(x, y, w_j) \approx \nabla \mathcal{L}(x, y, w_j)^T \Delta w_j + \frac{1}{2} \Delta w_j^T \mathbf{H} \Delta w_j \quad (3.2)$$

Where $\mathbf{H}$ is the Hessian $\nabla^2 \mathcal{L}(x, y, w_j)$. The authors then ignore the first term by asserting that the gradient $\nabla \mathcal{L}(x, y, w_j)$ is 0 for converged pretrained models, which simplifies into the following.

$$\mathcal{L}(x, y, w_j + \Delta w_j) - \mathcal{L}(x, y, w_j) \approx \frac{1}{2} \Delta w_j^T \mathbf{H} \Delta w_j \quad (3.3)$$

Finally, the authors formulate the optimization of weigh rounding for layer j as the following Quadratic Unconstrained Binary Optimization (QUBO) problem.

$$\arg \min_{\Delta w_j} \Delta w_j^T \mathbf{H} \Delta w_j \quad (3.4)$$

The above problem is NP-hard. To solve this efficiently, AdaRound uses the local mean squared error alongside a regularizer as a proxy. The regularizer encourages the optimization variables to converge towards either 0 or 1.

Derivation

$$\arg \min_{\mathbf{V}} \|\mathbf{W}x - \hat{\mathbf{W}}x\|_F^2 + f_{\text{reg}}(\mathbf{V}) \quad (3.5)$$

The regularizer uses an annealed parameter β and is defined as follows.

$$f_{\text{reg}}(\mathbf{V}) = \sum_{i,j} 1 - |2h(\mathbf{V}_{i,j}) - 1|^\beta \quad (3.6)$$

The above objective is then optimized over

$$\hat{\mathbf{W}} = \text{scale} \cdot \text{clamp} \left(\left\lfloor \frac{\mathbf{W}}{\text{scale}} \right\rfloor + h(\mathbf{V}), q_{\min}, q_{\max} \right), \quad (3.7)$$

$$q_{\min} = -2^{b-1}, \quad (3.8)$$

$$q_{\max} = 2^{b-1} - 1. \quad (3.9)$$

The authors define h as a rectified sigmoid as follows, although it can be any differentiable function with a range of $h(\mathbf{V}_{i,j}) \in [0, 1]$.

$$h(\mathbf{V}_{i,j}) = \text{clamp}(\sigma(\mathbf{V}_{i,j})(\zeta - \gamma) + \gamma, 0, 1), \quad (3.10)$$

$$\zeta = 1.1, \quad (3.11)$$

$$\gamma = -0.1. \quad (3.12)$$

The use of a rectified sigmoid is justified as it has non-vanishing gradients near 0 and 1, and helps the learning process as they previously used a regularizer to encourage $h(\mathbf{V}_{i,j})$ to move to either 0 or 1.

3.1.3 AdaRound Algorithm

We implement AdaRound in Algorithm 3.

Algorithm 3 AdaRound

Require: $\mathbf{W}$, X (calibration data), t (number of iterations), b (target bit width), β (regularization parameter)

Ensure: Quantized model $\mathbf{W}_q$, scales

- 1: $x_{in} \leftarrow \text{CACHEACTIVATIONS}(\mathbf{W}, X)$ ▷ Forward pass on calibration data to cache input activations
 - 2: **for** w_j in $\mathbf{W}$ **do**
 - 3: $q_{\min} \leftarrow -2^{b-1}$
 - 4: $q_{\max} \leftarrow 2^{b-1} - 1$
 - 5: $\text{scales}_j \leftarrow \frac{\max(w_j)}{q_{\max}}$ ▷ Store scale for output
 - 6: $\mathbf{V}_j \leftarrow h^{-1}(w_j - \lfloor w_j \rfloor)$ ▷ Initialize $\mathbf{V}_j$ so that it corresponds to the fraction of each weight
 - 7: **for** $i \in \{0 \dots t\}$ **do**
 - 8: $\beta \leftarrow \text{ANNEAL}(\beta)$
 - 9: $\hat{w}_j \leftarrow \text{clamp}\left(\lfloor \frac{w_j}{\text{scales}_j} \rfloor + h(\mathbf{V}_j), q_{\min}, q_{\max}\right)$
 - 10: $\mathcal{L} \leftarrow \|w_j x_{in} - \hat{w}_j x_{in}\|_F^2 + \sum_{i,j} (1 - |2h(\mathbf{V}_{i,j}) - 1|^\beta)$
 - 11: $\mathbf{V} \leftarrow \mathbf{V} - \nabla \mathcal{L}$ ▷ via autograd
 - 12: **end for**
 - 13: $\mathbf{W}_{Q_{i,j}} \leftarrow \begin{cases} \lceil w_{i,j} \rceil, & \text{where } h(\mathbf{V}_{i,j}) \geq 0.5 \\ \lfloor w_{i,j} \rfloor, & \text{otherwise} \end{cases}$ ▷ Finalize quantization
 - 14: **end for**
 - 15: **return** $\mathbf{W}_Q$, scales
-

3.1.4 Empirical Results

3.1.5 Limitations

Report
AdaRound
results

Two things
here: 1) zero
gradient as-
sumption, 2)
data leakage

Chapter 4

Proposed Method

In this chapter, we describe our two contributions: an inversion method to reconstruct representations of training data in AdaRound, and a proposed gradient-aware adaptive rounding method.

4.1 AdaRound Inversion Method

In this section, we will report our novel inversion method (also previously published as [20]) to reconstruct representations of the calibration data used by AdaRound, given access to a model before and after AdaRound quantization.

4.1.1 Formulation of Leakage Channel

AdaRound’s objective function is expected to approach zero after training. Let us define the weight perturbation matrix as $\Delta W = W - \hat{W}$. The objective function can be rewritten as the following.

$$\|Wx - \hat{W}x\|_F^2 = \|\Delta Wx\|_F^2 \quad (4.1)$$

After optimization, we expect the following.

$$\|\Delta Wx\|_F^2 \approx 0 \iff \Delta Wx \approx 0 \quad (4.2)$$

We notice that $x \in \mathcal{N}(\Delta W)$. In other words, the training data x used to calibrate the AdaRound process lies within the null space of the weight perturbation matrix ΔW .

4.1.2 Method

An attacker with access to the original weights W and the quantized weights $\hat{W}$ can generate a sample $\hat{x}$ that lies within this null space. We can initialize a $\hat{x}$ from random noise and apply Stochastic Gradient Descent with the objective $\arg \min_{\hat{x}} \|\Delta W\hat{x}\|_2^2$, as described in Algorithm 4.

Algorithm 4 Proposed Inversion Method**Require:** $W, \hat{W}, \hat{x}_{\text{init}}$, Number of Iterations K , Learning Rate η

-
- 1: $\Delta W \leftarrow W - \hat{W}$
 - 2: $\hat{x} \leftarrow \hat{x}_{\text{init}}$
 - 3: **for** $k = 1$ **to** K **do**
 - 4: $\mathcal{L} \leftarrow \sum (\Delta W \tanh(\hat{x}))^2$ (Forward pass)
 - 5: $\frac{\partial \mathcal{L}}{\partial \hat{x}} \leftarrow \text{Autograd}(\mathcal{L}, \hat{x})$ (Backward pass via autograd [21])
 - 6: $\hat{x} \leftarrow \hat{x} - \eta \frac{\partial \mathcal{L}}{\partial \hat{x}}$ (Update $\hat{x}$)
 - 7: **end for**
 - 8: **return** $\tanh(\hat{x})$
-

Note that the objective can be rewritten as the following.

$$\|\Delta W \hat{x}\|_2^2 = (\Delta W \hat{x})^T (\Delta W \hat{x}) = \hat{x}^T (\Delta W^T \Delta W) \hat{x} \quad (4.3)$$

A simple optimization can therefore be made by computing the Gram matrix $G = \Delta W^T \Delta W$ once, then use it to calculate the objective as $\hat{x}^T G \hat{x}$ in each iteration. (See Algorithm 5)

Algorithm 5 Proposed Inversion with Optimization**Require:** $W, \hat{W}, \hat{x}_{\text{init}}$, Number of Iterations K , Learning Rate η

-
- 1: $\Delta W \leftarrow W - \hat{W}$
 - 2: $\hat{x} \leftarrow \hat{x}_{\text{init}}$
 - 3: $G \leftarrow \Delta W^T \Delta W$
 - 4: **for** $k = 1$ **to** K **do**
 - 5: $\mathcal{L} \leftarrow \sum (\tanh(\hat{x})^T G \tanh(\hat{x}))$ (Forward pass)
 - 6: $\frac{\partial \mathcal{L}}{\partial \hat{x}} \leftarrow \text{Autograd}(\mathcal{L}, \hat{x})$ (Backward pass via autograd [21])
 - 7: $\hat{x} \leftarrow \hat{x} - \eta \frac{\partial \mathcal{L}}{\partial \hat{x}}$ (Update $\hat{x}$)
 - 8: **end for**
 - 9: **return** $\tanh(\hat{x})$
-

4.2 Gradient-Aware Adaptive Rounding

To reconcile AdaRound’s assumption that the gradient term in Equation 3.2 can be ignored, and Central Flows’ evidence that the gradient is nonzero even for pretrained converged models, we propose a modified version of AdaRound to incorporate awareness of the local gradient.

In order to minimize

$$\mathbb{E} \left[g^{(w)^T} \Delta W \right], \quad (4.4)$$

we can attempt to force ΔW to have the same sign as $-g^{(w)^T}$. We approximate this by finetuning a model on a small amount of training data, and compute the Exponential Moving Average (EMA) of the model during each iteration. We then use $\text{sign}(W'_{\text{avg}} - W)$,

the sign of the difference between the weights before and after finetuning, as a proxy to $-\text{sign}(g^{(w)^T})$. Finally, we add the Binary Cross Entropy (BCE) between the current rounding directions and $\text{sign}(W'_{\text{avg}} - W)$ as a penalty term to AdaRound's optimization objective.

Algorithm 6 Gradient-Aware Adaptive Rounding (Finetuning)

Require: $\mathbf{W}$, X (calibration data), η (learning rate), B (batch size)

Ensure: EMA of models during finetuning W'_{avg}

```

1:  $\mathbf{W}' \leftarrow \mathbf{W}$ 
2:  $\mathbf{W}'_{\text{avg}} \leftarrow \mathbf{W}$ 
3:  $N \leftarrow \frac{X}{B}$  ▷ Number of Iterations
4:  $\alpha \leftarrow \frac{2}{N+1}$  ▷ EMA Smoothing Factor
5: for each batch  $x_b$  in  $X$  do
6:    $\hat{x}_b \leftarrow \mathbf{W}'(x_b)$  ▷ Forward pass
7:    $\mathcal{L} \leftarrow \text{CrossEntropyLoss}(\hat{x}_b, x_b)$ 
8:    $\mathbf{W}' \leftarrow \mathbf{W}' - \eta \cdot \nabla \mathcal{L}$  ▷ Backward pass
9:    $\mathbf{W}'_{\text{avg}} \leftarrow \alpha \cdot \mathbf{W}'_{\text{avg}} + (1 - \alpha) \cdot \mathbf{W}'$  ▷ Update EMA
10: end for
11: return  $\mathbf{W}'_{\text{avg}}$ 

```

Algorithm 7 Gradient-Aware Adaptive Rounding (Quantization)

Require: $\mathbf{W}$, $\mathbf{W}'_{\text{avg}}$, X (calibration data), t (number of iterations), b (target bit width), β (regularization parameter), ψ (BCE loss multiplier)

Ensure: Quantized model $\mathbf{W}_Q$, scales

```

1:  $x_{in} \leftarrow \text{CACHEACTIVATIONS}(\mathbf{W}, X)$  ▷ Forward pass on calibration data to cache input activations
2: for  $w_j$  in  $\mathbf{W}$  do
3:    $\mathbf{D} \leftarrow \text{SIGN}(\mathbf{W}'_{\text{avg}} - \mathbf{W})$ 
4:    $q_{\min} \leftarrow -2^{b-1}$ 
5:    $q_{\max} \leftarrow 2^{b-1} - 1$ 
6:    $\text{scales}_j \leftarrow \frac{\max(w_j)}{q_{\max}}$  ▷ Store scale for output
7:    $\mathbf{V}_j \leftarrow h^{-1}(w_j - \lfloor w_j \rfloor)$  ▷ Initialize  $\mathbf{V}_j$  so that it corresponds to the fraction of each weight
8:   for  $i \in \{0 \dots t\}$  do
9:      $\beta \leftarrow \text{ANNEAL}(\beta)$ 
10:     $\hat{w}_j \leftarrow \text{clamp}\left(\lfloor \frac{w_j}{\text{scales}_j} \rfloor + h(\mathbf{V}_j), q_{\min}, q_{\max}\right)$ 
11:     $\mathcal{L} \leftarrow \underbrace{\psi \cdot \text{BCE}(h(\mathbf{V}_j), \mathbf{D})}_{\text{Proposed addition}} + \|w_j x_{in} - \hat{w}_j x_{in}\|_F^2 + \sum_{i,j} (1 - |2h(\mathbf{V}_{i,j}) - 1|^\beta)$ 
12:     $\mathbf{V} \leftarrow \mathbf{V} - \nabla \mathcal{L}$  ▷ via autograd
13:  end for
14:   $\mathbf{W}_{Q_{i,j}} \leftarrow \begin{cases} \lceil w_{i,j} \rceil, & \text{where } h(\mathbf{V}_{i,j}) \geq 0.5 \\ \lfloor w_{i,j} \rfloor, & \text{otherwise} \end{cases}$  ▷ Finalize quantization
15: end for
16: return  $\mathbf{W}_Q$ , scales

```

Chapter 5

Evaluation

In this chapter, we separately evaluate the AdaRound inversion method, and the proposed gradient-aware adaptive rounding method.

5.1 AdaRound Inversion Method

5.1.1 Experiment Setup

We trained MLPs on these datasets: Olivetti Faces [22], Chest X-Ray Images (Pneumonia) [23], and the Weizmann Horse Database [24], on a single computer (CPU: Intel Xeon w9-3495X, GPU: 1x NVIDIA GeForce RTX 4090). We perform 500 iterations of AdaRound quantization using the Adam optimizer, then obtain ΔW , the difference between the quantized and the original model. For each model, we generate 10 random samples from a Gaussian distribution, then train for 3000 iterations using the Adam optimizer. We report the SSIM (Structural Similarity Index Measure), PSNR (Peak Signal-to-Noise ratio), and Normalized RMSE (Root Mean Squared Error) between the mean of the calibration data and the mean of 10 generated samples before and after our inversion method in Table 5.1. We also report a visualization of the mean calibration sample vs. the mean generated sample in Figures 5.1, 5.2, and 5.3.

5.1.2 Results

Table 5.1: Metrics from Random Initialization vs. Our Proposed Method

	SSIM $\uparrow$	PSNR $\uparrow$	NRMSE $\downarrow$
Olivetti Faces (Before)	0.00	3.50	1.16
Olivetti Faces (After)	0.14 (+0.14)	12.83 (+9.33)	0.40 (-0.76)
Chest X-Ray (Before)	0.00	2.61	1.10
Chest X-Ray (After)	0.07 (+0.07)	10.13 (+7.52)	0.46 (-0.64)
Horse DB (Before)	0.00	2.60	1.11
Horse DB (After)	0.07 (+0.07)	10.15 (+7.55)	0.47 (-0.64)

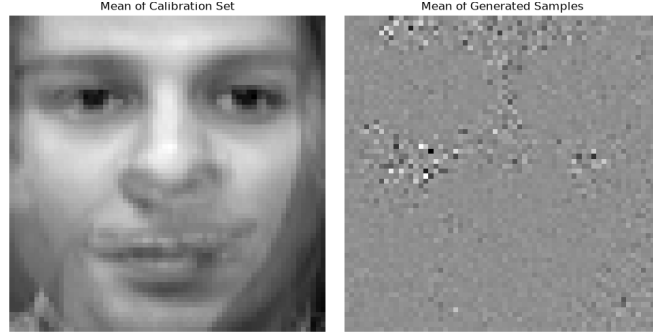


Fig. 5.1: Results on Olivetti Faces

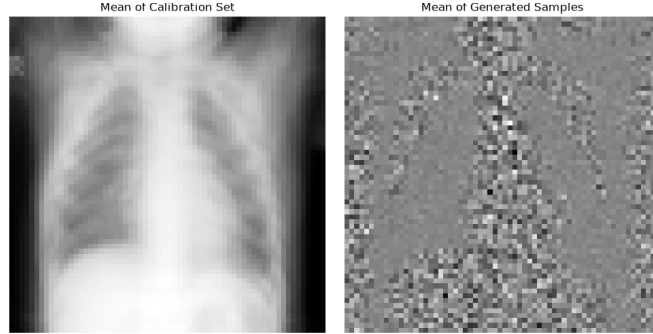


Fig. 5.2: Results on Chest X-Ray Images (Pneumonia)

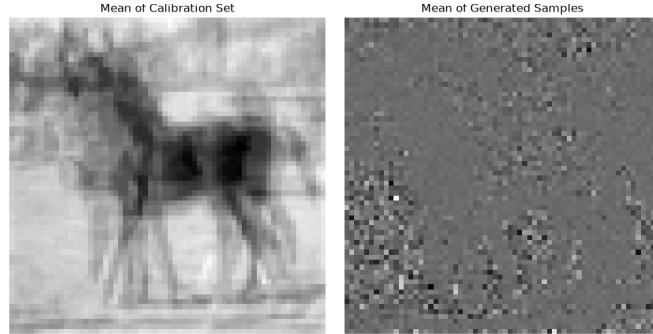


Fig. 5.3: Results on Weizmann Horse Database

We notice an increase in SSIM and PSNR, and a decrease in Normalized RMSE, implying that the mean of the generated samples resembles the mean of the calibration samples to some extent.

5.1.3 Ablation Study

We perform an ablation study by applying our inversion method to round-to-nearest (RTN) quantization. Since RTN does not use calibration data, we compare our method's samples to the mean of the training data instead. We report the same metrics as the

main experiment (SSIM, PSNR, and Normalized RMSE) in Table 5.2 and a visualization in Figures 5.4, 5.5, and 5.6.

Table 5.2: Results from Ablation Study

	SSIM $\uparrow$	PSNR $\uparrow$	NRMSE $\downarrow$
Olivetti Faces (Before)	0.00	2.89	1.12
Olivetti Faces (After)	0.00	3.90 (+1.01)	1.00 (-0.12)
Chest X-Ray (Before)	0.00	2.77	1.11
Chest X-Ray (After)	0.00	3.64 (+0.87)	1.00 (-0.11)
Horse DB (Before)	0.00	2.57	1.10
Horse DB (After)	0.00	3.38 (+0.81)	1.00 (-0.10)

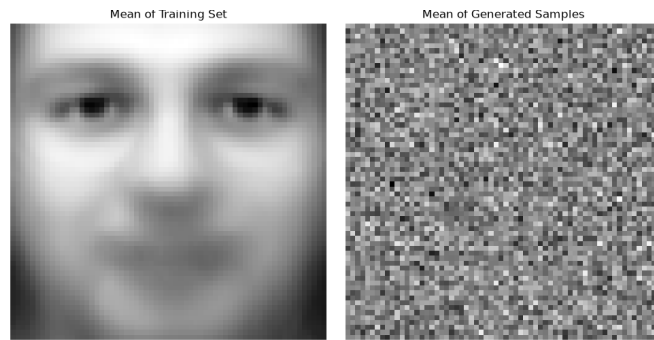


Fig. 5.4: Ablation Study Results on Olivetti Faces

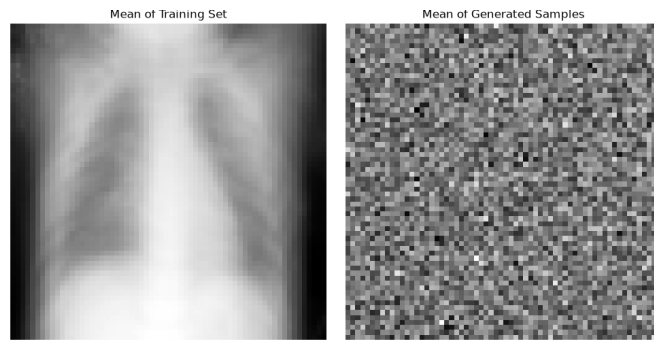


Fig. 5.5: Ablation Study Results on Chest X-Ray Images (Pneumonia)

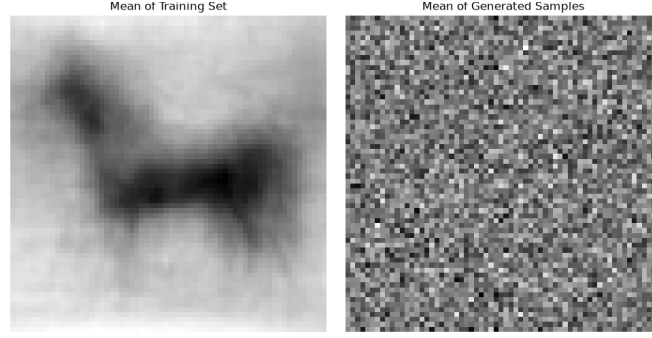


Fig. 5.6: Ablation Study Results on Weizmann Horse Database

We notice 0.00 SSIM on all datasets, implying that the generated samples do not look like the calibration data at all.

5.1.4 Discussion

Main Experiment

From the metrics, we observe an increase in SSIM and PSNR, and a decrease in Normalized RMSE after training. In the images, we can see the silhouettes and outlines of the calibration set, although it is quite difficult to make out. From the results, we can conclude that the ΔW of a model quantized using AdaRound does contain information about the calibration data.

Ablation Study

Interestingly, we see a slight improvement in PSNR and NRMSE in the ablation study. However, crucially the SSIM shows that round-to-nearest quantization is not vulnerable to our method. In other words, our method indeed extracts information about the calibration data in data-dependent quantization methods.

Overall

We demonstrate that AdaRound encodes information from the calibration data into the weight perturbation matrix. By targeting the null space of this matrix, our proposed method generates samples that resembles outlines of the calibration data. While the reconstructed data only represents the mean of the calibration data rather than exact instances, the leakage is significant enough to warrant reconsideration of the security of data-dependent PTQ methods.

5.1.5 Limitations

Low SSIM +
can only re-
cover means
of calibra-
tion data

5.2 Gradient-Aware Adaptive Rounding

5.2.1 Experiment Setup

We evaluate the proposed method on the same computer as described in Section 5.1.1. We report the perplexity of our two language models (TinyStories v1 and TinyStories v2) in Table 5.3.

5.2.2 Results

We observe neutral results using the proposed method.

Table 5.3: Perplexity Evaluation of Proposed Method

Type of Quantization	TinyStories V1	TinyStories V2
None (Unquantized Model)	11.9006	8.0994
Round-to-Nearest	11.9219	8.1224
AdaRound	11.9216	8.1176
Proposed Method	11.9166	8.1207

5.2.3 Ablation Study

We report an ablation study by ablating two possible elements of the proposed method: AdaRound and EMA. Ablating EMA means we use final finetuned model weights instead of the EMA of model weights during all iterations. Ablating AdaRound means we steer the rounding using only the sign of the weight perturbation matrix, as described in Algorithm 8.

Table 5.4: Perplexity Evaluation of Ablation Study

AdaRound with BCE Loss	EMA	TinyStories V1	TinyStories V2
✓	✓	11.9166	8.1546
✓	×	11.9216	8.1179
×	✓	11.9166	8.1546
×	×	11.9050	8.1084

Algorithm 8 Gradient-based Steering without AdaRound

Require: $\mathbf{W}$ (unquantized weights), $\mathbf{W}'$ (calibration model), b (target bit width)**Ensure:** Quantized model $\mathbf{W}_q$

```

1: function CUSTOMROUND( $\mathbf{W}, \mathbf{W}'$ )
2:    $\mathbf{W}_{i,j} \leftarrow \begin{cases} \lceil \mathbf{W}_{i,j} \rceil, & \text{where } \mathbf{W}_{i,j} < \mathbf{W}'_{i,j} \\ \lfloor \mathbf{W}_{i,j} \rfloor, & \text{otherwise} \end{cases}$ 
3:   return  $\mathbf{W}$ 
4: end function
5:  $q_{\min} \leftarrow -2^{b-1}$ 
6:  $q_{\max} \leftarrow 2^{b-1} - 1$ 
7:  $\text{scale} \leftarrow \frac{\max(\mathbf{W})}{q_{\max}}$ 
8:  $\mathbf{W}_q \leftarrow \text{clamp}\left(\text{CUSTOMROUND}\left(\frac{\mathbf{W}}{\text{scale}}\right), q_{\min}, q_{\max}\right)$ 
9: return  $\mathbf{W}_q, \text{scale}$ 

```

5.2.4 Discussion

5.2.5 Limitations

Chapter 6

Conclusion

Bibliography

- [1] Stuart Geman, Elie Bienenstock, and René Doursat. Neural networks and the bias/variance dilemma. *Neural Computation*, Vol. 4, No. 1, pp. 1–58, 1992.
- [2] Brady Neal, Sarthak Mittal, Aristide Baratin, Vinayak Tantia, Matthew Scicluna, Simon Lacoste-Julien, and Ioannis Mitliagkas. A Modern Take on the Bias-Variance Tradeoff in Neural Networks, December 2019.
- [3] Mikhail Belkin, Daniel Hsu, Siyuan Ma, and Soumik Mandal. Reconciling modern machine learning practice and the bias-variance trade-off. *Proceedings of the National Academy of Sciences*, Vol. 116, No. 32, pp. 15849–15854, August 2019.
- [4] Trevor Hastie, Robert Tibshirani, and J. H. Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer, New York, second edition, 2009.
- [5] Danny Hernandez, Tom Brown, Tom Conerly, Nova DasSarma, Dawn Drain, Sheer El-Showk, Nelson Elhage, Zac Hatfield-Dodds, Tom Henighan, Tristan Hume, Scott Johnston, Ben Mann, Chris Olah, Catherine Olsson, Dario Amodei, Nicholas Joseph, Jared Kaplan, and Sam McCandlish. Scaling Laws and Interpretability of Learning from Repeated Data, May 2022.
- [6] Johannes Gerstmayr, Peter Manzl, and Michael Pieber. Multibody models generated from natural language. *Multibody System Dynamics*, Vol. 62, pp. 249–271, January 2024.
- [7] C. E. Shannon. A mathematical theory of communication. *The Bell System Technical Journal*, Vol. 27, No. 3, pp. 379–423, 1948.
- [8] B. Ebanks, P.K. Sahoo, and W. Sander. *Characterizations of Information Measures*. World Scientific Publishing Company, 1998.
- [9] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 Technical Report, May 2025.
- [10] Samuel L. Smith, Benoit Dherin, David G. T. Barrett, and Soham De. On the Origin of Implicit Regularization in Stochastic Gradient Descent, 2021.
- [11] Ronen Eldan and Yuanzhi Li. TinyStories: How Small Can Language Models Be and Still Speak Coherent English?, May 2023.
- [12] Sid Black, Leo Gao, Phil Wang, Connor Leahy, and Stella Biderman. GPT-Neo: Large scale autoregressive language modeling with meshtensorflow. Zenodo, October 2021.
- [13] Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. 2019.

- [14] Taku Kudo and John Richardson. SentencePiece: A simple and language independent subword tokenizer and detokenizer for Neural Text Processing, August 2018.
- [15] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. LLaMA: Open and Efficient Foundation Language Models, February 2023.
- [16] F. Jelinek, R. L. Mercer, L. R. Bahl, and J. K. Baker. Perplexity—a measure of the difficulty of speech recognition tasks. *The Journal of the Acoustical Society of America*, Vol. 62, No. S1, pp. S63–S63, December 1977.
- [17] Markus Nagel, Marios Fournarakis, Rana Ali Amjad, Yelysei Bondarenko, Mart van Baalen, and Tijmen Blankevoort. A White Paper on Neural Network Quantization, June 2021.
- [18] S. Kullback and R. A. Leibler. On Information and Sufficiency. *The Annals of Mathematical Statistics*, Vol. 22, No. 1, pp. 79–86, March 1951.
- [19] Markus Nagel, Rana Ali Amjad, Mart van Baalen, Christos Louizos, and Tijmen Blankevoort. Up or Down? Adaptive Rounding for Post-Training Quantization, June 2020.
- [20] Piye Liu, Hiroshi Nakamura, and Hideki Takase. An Inversion Method to Reconstruct Training Data Representations in AdaRound Quantization. In *Summer United Workshops on Parallel, Distributed and Cooperative Processing*. Information Processing Society of Japan, August 2026.
- [21] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An Imperative Style, High-Performance Deep Learning Library, December 2019.
- [22] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, Vol. 12, No. 85, pp. 2825–2830, 2011.
- [23] Daniel S. Kermany, Michael Goldbaum, Wenjia Cai, Carolina C.S. Valentim, Huiying Liang, Sally L. Baxter, Alex McKeown, Ge Yang, Xiaokang Wu, Fangbing Yan, Justin Dong, Made K. Prasadha, Jacqueline Pei, Magdalene Y.L. Ting, Jie Zhu, Christina Li, Sierra Hewett, Jason Dong, Ian Ziyar, Alexander Shi, Runze Zhang, Lianghong Zheng, Rui Hou, William Shi, Xin Fu, Yaou Duan, Viet A.N. Huu, Cindy Wen, Edward D. Zhang, Charlotte L. Zhang, Oulan Li, Xiaobo Wang, Michael A. Singer, Xiaodong Sun, Jie Xu, Ali Tafreshi, M. Anthony Lewis, Huimin Xia, and Kang Zhang. Identifying Medical Diagnoses and Treatable Diseases by Image-Based Deep Learning. *Cell*, Vol. 172, No. 5, pp. 1122–1131.e9, February 2018.
- [24] E. Borenstein, E. Sharon, and Shimon Ullman. Combining top-down and bottom-up segmentation. Vol. 2004, pp. 46–46, July 2004.